

Séance 1 : Du Batch au Streaming

Owner	 louis reynouard
Tags	

Contexte et prérequis

Ce cours fait suite à Spark Core. Vous avez construit un pipeline batch sur le dataset NYC Taxi, une architecture Medallion Bronze/Silver/Gold, et un modèle K-Means sauvegardé dans votre Volume Unity Catalog.

Vérification avant de commencer : votre modèle K-Means doit être accessible dans votre Volume depuis Spark Core S5. La façon de le charger dépend de comment vous l'avez sauvegardé. Si vous avez utilisé `pipeline.save(chemin)`, chargez-le avec `PipelineModel.load(chemin)`. Si vous avez sauvegardé le modèle K-Means directement avec `model.save(chemin)` sans Pipeline, chargez-le avec `KMeansModel.load(chemin)` — dans ce cas, vous devrez appliquer votre `VectorAssembler` manuellement avant d'appeler `transform()`. Vérifiez dès maintenant que le chargement fonctionne dans une cellule test. Si ça échoue, reprenez la Séance 5 de Spark Core avant d'aller plus loin.

1. Pourquoi le batch ne suffit pas

Votre pipeline Spark Core tourne sur des fichiers historiques. Il suppose que les données sont déjà là quand le job démarre. Cette hypothèse casse dès qu'on se confronte à des situations concrètes.

Trois exemples dans le domaine taxi :

1. **Détection d'anomalie.** Un trajet enregistre 0 km pour 180 dollars. En batch quotidien, vous le détectez le lendemain matin dans le rapport. En streaming, vous le détectez dans les 30 secondes suivant la transaction.

2. **Surcharge de zone.** JFK Airport voit soudain 3x plus de demandes (vol international retardé). En batch, vous l'apprenez ce soir. En streaming, vous pouvez déclencher une alerte ou une redistribution de flotte immédiatement.
3. **Monitoring de SLA.** Votre contrat impose qu'aucun client ne dépasse 10 minutes d'attente. Le batch ne peut pas surveiller ça. Le streaming si.

Le batch reste pertinent pour l'entraînement de modèles, le reporting mensuel, les analyses profondes. Le streaming prend le relais quand la latence de décision compte.

2. Temps réel vs pseudo temps réel

La distinction est souvent mal comprise.

Le vrai temps réel (hard real-time) exige une réponse garantie en quelques millisecondes avec une borne supérieure stricte. C'est le domaine des systèmes embarqués, du contrôle industriel, de la finance haute fréquence. Spark n'est pas conçu pour ça.

Le pseudo temps réel (near real-time) tolère une latence de quelques secondes à quelques minutes. C'est le domaine de Spark Structured Streaming. Pour la grande majorité des cas métier (détection de fraude, monitoring opérationnel, alertes sur KPI), cette latence est acceptable.

Dans ce cours, "temps réel" désigne du pseudo temps réel avec Spark Structured Streaming.

3. Spark Structured Streaming : la mécanique fondamentale

Le modèle mental : une table infinie

Spark Structured Streaming repose sur une idée simple : un flux de données est une table qui grandit indéfiniment. À chaque micro-batch, Spark lit les nouvelles lignes arrivées depuis la dernière exécution, applique les transformations, et écrit les résultats.

Vous utilisez exactement la même API DataFrame que dans Spark Core. La différence est uniquement dans la lecture et l'écriture.

```
# Spark Core (batch)
df = spark.read.format("delta").load("/chemin/vers/table")

# Spark Structured Streaming
df = spark.readStream.format("delta").load("/chemin/vers/table")
```

Les transformations que vous connaissez (filtres, jointures, `withColumn`) fonctionnent sans modification. Ce qui change, c'est le comportement d'exécution : au lieu de tourner une fois et s'arrêter, le job tourne en boucle jusqu'à ce que vous l'arrêtiez.

Les composants d'un job streaming

La source : d'où viennent les données. Dans ce cours, on utilise Auto Loader (`cloudFiles`), qui surveille un dossier dans un Volume Unity Catalog et détecte automatiquement les nouveaux fichiers sans les lister manuellement.

Les transformations : votre logique métier. Identique au batch.

Le sink : où vont les résultats. On écrit en tables Delta avec `.writeStream` à la place de `.write`.

Le trigger : cadence d'exécution

```
# Micro-batch toutes les 30 secondes
.trigger(processingTime="30 seconds")

# Traite tout ce qui est disponible puis s'arrête : utile pour les tests
.trigger(availableNow=True)

# Micro-batch aussi vite que possible (déconseillé sur compte
```

```
gratuit)  
.trigger(processingTime="0 seconds")
```



Sur le compte gratuit Databricks, le compute est exclusivement serverless. Or serverless ne supporte pas `processingTime` : lancer un job avec `.trigger(processingTime="30 seconds")` lève immédiatement l'erreur `INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED`.

Le seul trigger compatible est `availableNow=True`. Concrètement, cela signifie que le job ne tourne pas en boucle : il traite tout ce qui est disponible dans la source au moment du lancement, écrit les résultats, puis s'arrête tout seul.

Pour simuler un comportement proche du streaming continu, on utilise une boucle Python qui alterne le simulateur et le job streaming. Dans ce cours, on travaille sur 10 itérations avec 20 secondes de pause entre chaque, ce qui donne :

```
import time
import threading

for i in range(10):
    # Lancer le simulateur pour un seul batch puis ar
    # rêt automatique
    sim = TaxiStreamSimulator(
        output_path=SIM_OUTPUT_PATH,
        rate=20,
        interval_seconds=0, # pas de pause interne,
        on gère nous-mêmes
        mode="normal",
        duration_batches=1, # un seul batch par itér
        ation
        seed=i # seed différent à chaqu
        e tour pour de la variété
    )
    t = threading.Thread(target=sim.run)
    t.start()
    t.join() # attend que le fichier Parquet soit éc
    rit
```

```

# Lancer le job streaming sur les nouveaux fichiers
rs
query = (
    df_stream.writeStream
        .format("delta")
        .outputMode("append")
        .option("checkpointLocation", checkpoint_path)
    h)
    .trigger(availableNow=True)
    .toTable("workspace.tp_spark_XYZ.bronze_stream_taxi")
)
query.awaitTermination() # attend que le job soit terminé

print(f"Itération {i+1}/10 terminée")
time.sleep(20) # pause avant la prochaine itération
ion

```

La Spark UI Structured Streaming fonctionne toujours : chaque appel produit une entrée dans l'historique des queries. La Batch Duration mesurée séance par séance reste un indicateur valide pour le challenge de volume de la Séance 5.

Le checkpoint : la mémoire du job

Le checkpoint est le mécanisme qui permet au job de reprendre là où il s'est arrêté après un crash ou un redémarrage. Sans checkpoint, le job relit tout depuis le début.

```

.option("checkpointLocation", "/Volumes/workspace/tp_spark_XYZ/checkpoints/bronze_stream")

```

Un checkpoint est lié à une query spécifique. Si vous modifiez votre schéma ou votre logique, vous devrez souvent supprimer et recréer le checkpoint. Stockez-

les dans votre Volume, pas en mémoire locale.

4. Auto Loader

Auto Loader (`cloudFiles`) est la solution recommandée sur Databricks pour ingérer des fichiers en continu. Il surveille un répertoire, détecte automatiquement les nouveaux fichiers, et ne les traite qu'une seule fois grâce au checkpoint.

Deux options à configurer obligatoirement : `cloudFiles.format` (le format des fichiers à surveiller) et `cloudFiles.schemaLocation` (un chemin où Auto Loader mémorise le schéma inféré la première fois, pour éviter de rescanner tous les fichiers à chaque redémarrage).

Le chemin passé à `.load()` est le répertoire à surveiller. Auto Loader détecte les nouveaux fichiers au fil des micro-batches sans les relire.

5. Les modes de sortie

append : seules les nouvelles lignes sont écrites. C'est le mode adapté aux transformations sans agrégation stateful, et le plus courant pour l'ingestion.

complete : la table résultat est entièrement réécrite à chaque micro-batch. Utilisé uniquement avec les agrégations globales. Très coûteux : à éviter sur compte gratuit.

update : seules les lignes modifiées sont écrites. Le bon compromis pour les agrégations stateful avec Delta Lake.

6. Contraintes du compte Databricks gratuit

Le compte gratuit impose des limites concrètes :

- Cluster unique, single-node. Pas de distribution réelle, Spark tourne sur une seule machine.
- Timeout automatique : le cluster s'arrête après environ 2h d'inactivité.
- RAM limitée : typiquement 15 à 30 Go selon le plan.

Un job qui tourne 4h risque d'être tué par le timeout cluster.

Adaptations à appliquer dans tous vos notebooks :

- Trigger minimum 30 secondes.
 - Simulations petites (10 à 50 trajets par batch).
 - `trigger(availableNow=True)` pour les tests et validations.
 - Checkpoints dans un Volume persistant.
-

TP Séance 1 : Premier pipeline streaming

Ce qui vous est donné

Le script `taxi_simulator.py` à déposer dans votre Volume. Le mode imposé pour cette séance est `normal`, sans données tardives. Tous les autres paramètres (rate, interval, duration) sont votre décision.

Ce que vous devez construire

Le simulateur configuré et lancé dans un thread daemon. Chaque paramètre doit être justifié dans une cellule Markdown : pourquoi ce rate, pourquoi cet interval, combien de batches et pourquoi.

Un pipeline Bronze streaming qui surveille le dossier de sortie du simulateur via Auto Loader, ajoute les colonnes techniques (`ingestion_timestamp`, `source_file`), et écrit en table Delta en mode `append`. Le trigger et le checkpoint sont votre choix, à justifier.

Une preuve d'exécution. Pendant que le job tourne, ouvrez la Spark UI onglet Structured Streaming et relevez le nombre de batches traités et le nombre de lignes par batch. Cette capture ou ce log doit figurer dans le notebook.

Une validation de cohérence de schéma entre votre table `bronze_stream_taxi` et la table `bronze_taxi_trips` de Spark Core. Si des différences existent, commentez-les.

Contraintes techniques

Le checkpoint doit être dans un Volume persistant. Le trigger doit être justifié. Le job doit se terminer par un arrêt explicite. Toutes les occurrences de `XYZ` sont remplacées par votre trigramme.

Livrable : Notebook `S1_Ingestion_Stream_TRIGRAMME`

Le pipeline Bronze fonctionnel avec ses justifications. La preuve d'exécution (Spark UI ou logs). La validation de schéma commentée. Une cellule Markdown répondant à : pourquoi le mode `complete` est impraticable pour une table d'ingestion brute en streaming ? 5 lignes minimum, basées sur des arguments techniques.

Grille d'évaluation

Critère	Insuffisant	Attendu	Excellent
Checkpoint	Absent ou chemin local	Volume persistant	Volume + justification du chemin
Trigger	Valeur par défaut sans explication	Valeur adaptée aux contraintes compte gratuit	Valeur + mesure de l'impact observé
Cohérence schéma	Non vérifiée	Colonnes identiques à Spark Core	Identiques + explication des différences éventuelles
Monitoring	Aucune trace	Capture Spark UI ou logs	Spark UI + analyse du débit observé
Question théorique	Paraphrase du cours	Réponse technique précise	Réponse + exemple chiffré ou calcul

Un notebook sans cellules de documentation (Markdown) ne sera pas accepté.